

Scan authentication diagnosis

Confirmed cause

A read-only Gemini model-list request with the configured local credential reproduced HTTP **401**, **UNAUTHENTICATED**, reason **ACCESS_TOKEN_TYPE_UNSUPPORTED**. No image or generation prompt was sent. The credential has an **AQ.** prefix, not the standard Google API-key format supported by this integration. Its value is intentionally omitted.

The installed Google Generative AI SDK sends the supplied credential in the API-key header to the Gemini Developer API. It does not add a bearer authorization header. Neither Pl@ntNet, Perenual, nor the frontend user-session token supplies Gemini authentication.

The scan helper already resolved its key at request time before this fix. Its previous negative checks rejected a few token prefixes but accepted this incompatible credential. Thus import-time initialization was **not** the direct cause of the reported scan failure. The separate legacy AI endpoint did capture credentials at import time and use an older hard-coded model; that independent defect is now fixed too.

The inspection found no inherited Gemini key overriding the local value and no alternate Google key setting. Previously, dotenv resolved relative to the launch directory; environment loading now uses a stable backend-relative path while preserving deployment-variable precedence.

Data flow



Syntax error in text
mermaid version 12.0.0

Species enrichment and telemetry retrieval run concurrently. Missing Perenual data is optional; Gemini authentication is mandatory. Scan persistence starts after generation succeeds, although species-cache writes may occur earlier. No fabricated diagnosis is used to hide an authentication rejection.

Implementation

1. Centralize backend environment loading in [env.ts](#), shared by startup, auth configuration and database clients.
2. Validate mandatory Gemini and Pl@ntNet configuration in [provider-config.ts](#). Gemini validation enforces the standard Google API-key shape. This checks format only, not revocation, restrictions, quota or model access.
3. Execute [provider-startup.ts](#) before server dependencies initialize. Missing or malformed mandatory credentials produce a redacted, actionable message and exit status 1 before listening. Perenual remains optional.

4. Resolve credentials on each model creation in [gemini.ts](#), without module-level credential caching or automatic fallback to another credential source. Share model selection and the 45-second timeout across both endpoints; retain JSON output for scans and text output for legacy analysis.
5. Validate before external identification in [scan.ts](#). Sanitize authentication, quota and network errors in both this route and [ai.ts](#); provider authentication errors become service-configuration 503 responses, not user-session 401 responses.
6. Test real SDK headers with mocked HTTP, runtime key rotation, startup failure, legacy endpoint recovery, scan cache behavior and failure-before-persistence.

Operational remediation and verification limits

Replace the incompatible backend Gemini credential with a standard Gemini Developer API key from Google AI Studio, configure API restrictions/project access, and restart the backend. Do not paste a bearer token, service-account document or incompatible auth-key credential into the API-key setting. Existing deployment environment values take precedence over the local environment file.

The local secret was not modified. With the incompatible value still configured, startup is intentionally blocked rather than allowing every scan to fail at the last stage. A successful live diagnostic scan cannot be certified until a compatible authorized credential is provisioned. Automated tests use synthetic credentials and mocked providers; they do not establish live key validity or quota.

The backend build initially encountered a Windows engine-file lock. Retrying after the lock cleared successfully regenerated the declared Prisma 6.19.3 clients, compiled TypeScript, and packaged the PostgreSQL client. This also addressed the stale installed client encountered during the first test run; no Prisma upgrade or schema change was made for this fix.

Final verification: backend build **passed**; standalone TypeScript checking **passed**; full backend test suite **51 passed, 0 failed**. Backend source/test whitespace checks passed (Git reported only line-ending conversion warnings). Successful scan-route tests use mocked providers and storage; the live read-only probe reproduced the credential rejection, not a successful production diagnosis.